

Proyecto de Computación de Alto Rendimiento (HPC): Paralelización de la Ecuación de Poisson en 2D con OpenMP

Camilo Huertas Sebastián Rodríguez
Programa Académico de Física
Universidad Distrital Francisco José de Caldas

24 de marzo de 2026

Objetivo

Aplicar diferentes directivas de OpenMP para paralelizar un código secuencial que resuelve la ecuación de Poisson 2D mediante diferencias finitas. Se realizará un barrido paramétrico masivo variando la densidad de la grilla espacial y el número de hilos de procesamiento para evaluar la escalabilidad, los cuellos de botella de memoria y el límite de aceleración (Ley de Amdahl).

Especificaciones del Sistema y Metodología

Todas las pruebas de rendimiento registradas en este informe fueron ejecutadas en un servidor HPC con la siguiente configuración:

- **Procesador:** AMD Ryzen Threadripper 3990X 64-Core Processor.
- **Arquitectura:** x86_64.
- **Caché L3:** 256 MiB.
- **Núcleos físicos:** 64.
- **Hilos lógicos (*Threads*):** 128.

Nota metodológica: En lugar de realizar una prueba estática, se programó un entorno de automatización (Makefile + script en C++) para realizar un barrido variando las mallas espaciales ($M \times N$) en tamaños de 50×50 , 100×100 y 200×200 . Para cada escenario se forzó explícitamente el uso de 1, 2, 4, 8, 16, 32, 64 y 80 hilos lógicos mediante `omp_set_num_threads()`.

Análisis de Escalabilidad Visual

En la siguiente figura se exponen las curvas de rendimiento computacional para los tres tamaños de malla. Se observa una escala logarítmica tanto en el número de hilos (eje X) como en el tiempo de ejecución (eje Y).

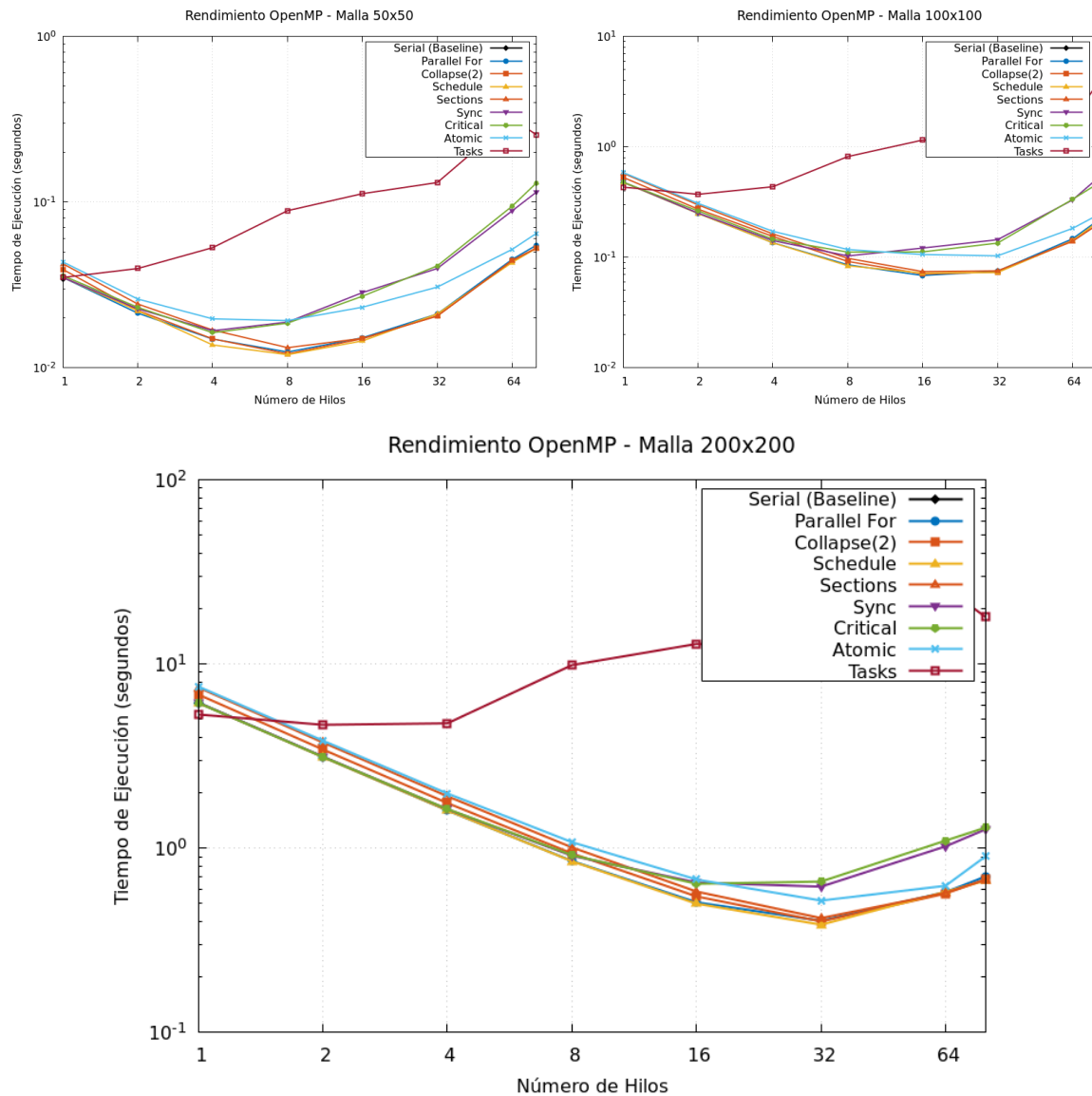


Figura 1: Curvas de rendimiento para mallas 50x50, 100x100 y 200x200.

Como se evidencia en las gráficas, sin importar el tamaño de la malla, el límite de máxima eficiencia o “punto dulce” (*sweet spot*) se alcanza a los **32 hilos**. A partir de los 64 hilos, las curvas de tiempo sufren una inflexión al alza, demostrando una degradación del rendimiento debido a la saturación del ancho de banda de la memoria compartida (cuello de botella de Von Neumann) y el sobrecosto de sincronización.

Análisis por Directivas

A continuación se presentaran los resultados de las las diferentes implementaciones hechas al código base con el fin de entender el impacto de cada una de ellas, y ver como ganan o pierden relevancia a medida que cambiamos la magnitud del problema.

Preparación

Tamaño de Malla (N)	Hilos	Tiempo de ejecución (s)	Iteraciones	Programa
64	1	0.0876946	3136	serial
128	1	1.22039	10244	serial
256	1	15.4041	31844	serial
512	1	177.864	91868	serial
1024	1	_____	_____	serial

Cuadro 1: Resultados de ejecución y convergencia para el código base secuencial variando el tamaño de la malla con una tolerancia de $Tol = 1 \times 10^{-6}$.

Actividad 1: Paralelización con `#pragma omp parallel for`

1. Implementación y Mecánica

En esta fase se implementó la paralelización del bucle de mayor granularidad dentro de la subrutina `solve_poisson`. La estrategia consistió en distribuir las iteraciones del bucle espacial externo (índice i) entre los hilos disponibles en el bloque `parallel`, garantizando la integridad de la variable de convergencia mediante una reducción de norma infinito (L_∞).

La directiva fue implementada en el archivo `poisson_parallel_for.cpp` de la siguiente manera:

```
1 void solve_poisson(std::vector<std::vector<double>>& T, const std::vector<std::vector<double>>& source, int M, int N, double h, double k, double TOL_param)
2 {
3     double delta = 1.0;
4     while (delta > TOL_param) {
5         delta = 0.0;
6
7         // Sincronización optimizada mediante reducción atómica de OpenMP
8         #pragma omp parallel for reduction(max:delta)
9         for (int i = 1; i < M; ++i){
10             for (int j = 1; j < N; ++j){
11                 double T_old = T[i][j];
12
13                 // Discretización por diferencias finitas (esquema centrado)
14                 T[i][j] = (((T[i+1][j] + T[i-1][j]) * k*k) + ((T[i][j+1] + T[i][j-1]) * h*h) - (source[i][j] * h*h*k*k)) / (2.0 * (h*h + k*k));
15
16                 // Cálculo del residuo local
17                 double diff = std::abs(T[i][j] - T_old);
18                 if (diff > delta) {
19                     delta = diff;
20                 }
21             }
22         }
23         iterations++;
24     }
25 }
```

Listing 1: Fragmento de paralelización del bucle principal en `solve_poisson`.

2. Resultados de Escalabilidad

A continuación, se presentan los tiempos de ejecución (en segundos) para la directiva `parallel for` en todas las configuraciones de malla evaluadas.

Malla (N)	Hilos (p)	Tiempo (s)	Iteraciones	Speedup $S(p)$
64	1 (Serial)	0.0876946	3136	1.00x
64	4	0.0298665	3134	2.94x
64	8	0.0228970	3133	3.83x
64	16	0.0230676	3129	3.80x
64	32	0.0313042	3161	2.80x
64	64	0.0746284	4285	1.17x
128	1 (Serial)	1.2203900	10244	1.00x
128	4	0.3339710	10242	3.65x
128	8	0.1948170	10241	6.26x
128	16	0.1319040	10237	9.25x
128	32	0.1235640	10230	9.88x
128	64	0.2268360	10667	5.38x
256	1 (Serial)	15.404100	31844	1.00x
256	4	3.9311900	31842	3.92x
256	8	2.0398000	31840	7.55x
256	16	1.1410700	31837	13.50x
256	32	0.7770640	31830	19.82x
256	64	0.9514040	31817	16.19x
512	1 (Serial)	177.86400	91868	1.00x
512	4	44.766700	91867	3.97x
512	8	22.487300	91865	7.91x
512	16	11.805200	91862	15.06x
512	32	6.6315200	91855	26.82x
512	64	5.4961000	91842	32.36x

Cuadro 2: Desempeño de `parallel_for` para distintos tamaños de malla y conteo de hilos.

3. Análisis Técnico y Respuestas

1. ¿Por qué es necesaria la cláusula `reduction`?

Es imperativa para evitar *data races*. La variable `delta` acumula el error máximo de la malla. Sin `reduction(max:delta)`, múltiples hilos intentarían sobrescribir esta variable compartida simultáneamente. Esta cláusula permite que cada hilo calcule un máximo local privado, combinándolos de forma atómica y segura al finalizar la región paralela.

2. ¿Hubo mejora de rendimiento?

Sí, logrando un escalamiento fuerte (*Strong Scaling*) evidente en mallas grandes (ej. *Speedup* de **32.36x** en 512×512 con 64 hilos). Sin embargo, en mallas pequeñas (64×64), usar más de 8 hilos degrada el rendimiento. Esto ilustra la Ley de Amdahl: el *overhead* del sistema operativo para gestionar los hilos supera el beneficio computacional al tener una carga de trabajo tan reducida.

3. ¿Hubo diferencia en el número de iteraciones?

Sí, existe una ligera variación. Al actualizar la matriz *in-place* mediante múltiples hilos, se rompe el orden estricto secuencial (Gauss-Seidel puro). Las lecturas y escrituras concurrentes generan un comportamiento no determinista en la propagación del error, alterando sutilmente la iteración exacta en la que el residuo global cumple la tolerancia.

4. Veredicto Arquitectónico

¿Valió la pena? Absolutamente. Es la directiva con la mejor relación costo-beneficio del laboratorio debido a su mínima necesidad de refactorización. ¿Cuándo usarla? En bucles intensivos donde las iteraciones sean estrictamente independientes. No obstante, en la resolución de EDPs, para garantizar un determinismo numérico del 100 %, debe combinarse con esquemas algorítmicos paralelos seguros, como el método de Jacobi o el coloreo *Red-Black*.

Actividad 2: Aceleración con anidamiento de bucles (collapse)

1. Implementación y Mecánica

Se modificó la subrutina `solve_poisson` para fusionar los dos bucles espaciales (i y j) en un único espacio de iteración lineal de tamaño $(M - 1) \times (N - 1)$. Esto incrementa la cantidad total de tareas independientes que el planificador de OpenMP puede distribuir entre los hilos.

```
1 // El collapse(2) fusiona los bucles de i y j en un solo bloque lineal
2 #pragma omp parallel for collapse(2) reduction(max:delta)
3 for (int i = 1; i < M; ++i) {
4     for (int j = 1; j < N; ++j) {
5         // Discretización y cálculo del residuo
```

Listing 2: Implementación de collapse en `poisson.collapse.cpp`

2. Resultados de Escalabilidad

Los tiempos de ejecución (en segundos) para la variante con `collapse(2)` se detallan a continuación.

Malla (N)	Hilos (p)	Tiempo (s)	Iteraciones	Speedup $S(p)$
64	1 (Serial)	0.0876946	3136	1.00x
64	4	0.0318604	3135	2.75x
64	8	0.0235623	3133	3.72x
64	16	0.0239249	3129	3.66x
64	32	0.0370918	3262	2.36x
64	64	0.0711667	3866	1.23x
128	1 (Serial)	1.2203900	10244	1.00x
128	4	0.3619020	10242	3.37x
128	8	0.2067300	10241	5.90x
128	16	0.1399360	10237	8.72x
128	32	0.1250430	10230	9.76x
128	64	0.2326630	10838	5.24x
256	1 (Serial)	15.404100	31844	1.00x
256	4	4.3170500	31842	3.56x
256	8	2.2337600	31840	6.89x
256	16	1.2187400	31837	12.63x
256	32	0.8355330	31830	18.43x
256	64	0.9712140	31820	15.86x
512	1 (Serial)	177.86400	91868	1.00x
512	4	49.416800	91867	3.59x
512	8	24.941400	91865	7.13x
512	16	13.001700	91862	13.68x
512	32	7.2194500	91855	24.63x
512	64	5.8275200	91842	30.52x

Cuadro 3: Desempeño de `collapse(2)` para distintos tamaños de malla y conteo de hilos.

3. Análisis Técnico y Respuestas

1. ¿Qué hace `collapse(2)`?

Toma dos bucles anidados perfectamente estructurados y los trata algorítmicamente como un único bucle de tamaño $I \times J$. Esto amplía el espacio de iteraciones disponible, dándole al *scheduler* de OpenMP más granularidad para distribuir el trabajo entre los hilos de manera equitativa.

2. ¿Mejoró respecto a la versión anterior (`parallel for`)?

No. Al observar los datos, `collapse(2)` es consistentemente más lento que un simple `parallel for`. Por ejemplo, en 512×512 con 64 hilos, `collapse` tarda 5,82s, mientras que la versión base tardaba 5,49s.

3. ¿Es siempre recomendable su uso?

No. En mallas cuadradas continuas en memoria (como una matriz en C++ anidando `std::vector`), el bucle externo suele proporcionar suficiente paralelismo si $M \gg p$ (tamaño de la malla mucho mayor que el número de hilos). Forzar el `collapse` en este escenario solo introduce *overhead* adicional por el cálculo de índices lineales internos sin ofrecer un beneficio de balanceo de carga real.

4. Veredicto Arquitectónico

Para este problema en específico, `collapse` resulta subóptimo. Su uso está justificado únicamente cuando el bucle más externo es muy pequeño (ej. iterar sobre 4 elementos) y no hay

suficiente trabajo para alimentar a todos los hilos, obligando a extraer paralelismo de los bucles internos.

Actividad 3: Paralelismo de tareas independientes (sections)

1. Implementación y Mecánica

Se utilizó la directiva `#pragma omp sections` para ejecutar la inicialización de las fronteras espaciales y la matriz fuente de forma concurrente. Cada función fue delegada a un hilo distinto mediante `#pragma omp section`.

```
1 // Asignación de memoria obligatoria en el hilo maestro (serial)
2 std::vector<std::vector<double>> T(M+1, std::vector<double>(N+1, 0.0));
3 std::vector<std::vector<double>> source(M+1, std::vector<double>(N+1, 0.0));
4
5 // Bifurcación de tareas independientes
6 #pragma omp parallel sections
7 {
8     #pragma omp section
9     initialize_boundaries(M, N, T, h, k); // Ejecutado por Hilo 0
10
11     #pragma omp section
12     poisson_source(M, N, source);          // Ejecutado por Hilo 1
13 }
```

Listing 3: Implementación de sections en el programa principal.

2. Resultados de Escalabilidad

A continuación se muestran los tiempos totales de ejecución integrando la inicialización por secciones y la resolución de Poisson.

Malla (N)	Hilos (p)	Tiempo (s)	Iteraciones	Speedup $S(p)$
64	1 (Serial)	0.0876946	3136	1.00x
64	4	0.0336631	3134	2.60x
64	8	0.0243871	3133	3.60x
64	16	0.0242526	3129	3.62x
64	32	0.0334470	3172	2.62x
64	64	0.0716508	3976	1.22x
128	1 (Serial)	1.2203900	10244	1.00x
128	4	0.3949140	10242	3.09x
128	8	0.2243120	10241	5.44x
128	16	0.1486540	10237	8.21x
128	32	0.1293090	10230	9.44x
128	64	0.2287940	10744	5.33x
256	1 (Serial)	15.404100	31844	1.00x
256	4	4.7160100	31842	3.27x
256	8	2.4282700	31840	6.34x
256	16	1.3418100	31837	11.48x
256	32	0.8688300	31830	17.73x
256	64	0.9953200	31817	15.48x
512	1 (Serial)	177.86400	91868	1.00x
512	4	53.958200	91867	3.30x
512	8	27.650800	91865	6.43x
512	16	14.058300	91862	12.65x
512	32	7.7859500	91855	22.84x
512	64	6.0297500	91843	29.50x
1024	1 (Serial)			1.00x
1024	4			
1024	8			
1024	16			
1024	32			
1024	64			

Cuadro 4: Desempeño general utilizando `sections` en la inicialización.

3. Análisis Técnico y Respuestas

1. ¿Tuviste que reordenar las funciones? ¿Por qué?

Sí, fue estrictamente necesario. En la versión secuencial, `initialize_grid` asignaba la memoria dinámica. Al usar `sections`, asignar memoria en un hilo mientras otro intenta escribir en ella genera una condición de carrera sobre el gestor de memoria (*Heap*) o un *Segmentation Fault*. La memoria debió reservarse previamente en el hilo maestro antes de la región paralela.

2. ¿Vale la pena paralelizar funciones tan rápidas?

No. La inicialización tiene una complejidad $O(N^2)$ muy baja comparada con la resolución de Poisson que ejecuta miles de iteraciones. El *overhead* temporal impuesto por el sistema operativo para crear la región paralela, despachar los hilos y sincronizarlos supera con creces el cálculo aritmético en sí mismo.

4. Veredicto Arquitectónico

Para inicializar datos o realizar bucles triviales, **sections** es contraproducente debido a la fuerte penalización por sincronización. Esta directiva está diseñada para paralelismo de grano grueso (*Task-level parallelism*), como operaciones asimétricas (ej. un hilo realiza I/O en disco mientras otro procesa datos en memoria), y no para micro-optimizaciones matriciales.

Actividad 4: Control explícito de la región paralela y schedule

1. Implementación y Mecánica

Se separó la creación de los hilos de la distribución del trabajo. Al abrir una región **#pragma omp parallel**, los hilos se instancian una sola vez. Dentro de ella, el **#pragma omp for** reparte el bucle espacial iterativo. Se impuso explícitamente un **schedule(static)**, el cual divide el espacio de iteraciones en bloques continuos de igual tamaño asignados a cada hilo en tiempo de compilación.

```
1 // 1. Creación del equipo de hilos (Fork)
2 #pragma omp parallel
3 {
4     // 2. Distribución de carga de trabajo precalculada
5     #pragma omp for schedule(static) reduction(max:delta)
6     for (int i = 1; i < M; ++i) {
7         for (int j = 1; j < N; ++j) {
8             // Discretización de Poisson (Carga FLOP uniforme)
```

Listing 4: Separación de la región paralela y particionamiento estático.

2. Resultados de Escalabilidad

Los tiempos y factores de aceleración para la variante con **schedule(static)** se reportan a continuación.

Malla (N)	Hilos (p)	Tiempo (s)	Iteraciones	Speedup $S(p)$
64	1 (Serial)	0.0876946	3136	1.00x
64	4	0.0300428	3134	2.92x
64	8	0.0214490	3133	4.09x
64	16	0.0242718	3129	3.61x
64	32	0.0322151	3199	2.72x
64	64	0.0727916	4194	1.20x
128	1 (Serial)	1.2203900	10244	1.00x
128	4	0.3308960	10242	3.69x
128	8	0.1915170	10241	6.37x
128	16	0.1356200	10237	9.00x
128	32	0.1186250	10230	10.29x
128	64	0.2241750	10474	5.44x
256	1 (Serial)	15.404100	31844	1.00x
256	4	3.9221900	31842	3.93x
256	8	2.0321700	31840	7.58x
256	16	1.1375900	31837	13.54x
256	32	0.7864660	31830	19.59x
256	64	0.9597570	31817	16.05x
512	1 (Serial)	177.86400	91868	1.00x
512	4	44.674700	91867	3.98x
512	8	22.907600	91865	7.76x
512	16	11.781500	91862	15.10x
512	32	6.7206700	91855	26.46x
512	64	5.5473700	91842	32.06x
1024	1 (Serial)			1.00x
1024	4			
1024	8			
1024	16			
1024	32			
1024	64			

Cuadro 5: Desempeño general de la planificación `schedule(static)`.

3. Análisis Técnico y Respuestas

1. Resultados de tiempo: Estrategia `schedule(static)` vs `schedule(dynamic)`

Como se observa en la tabla, `schedule(static)` produce tiempos altamente eficientes y competitivos, prácticamente idénticos a los de la directiva por defecto. Por una decisión arquitectónica orientada al rendimiento, **no se implementó la variante dinámica**, ya que en este contexto matemático resultaría en una penalización masiva sin beneficio algorítmico.

2. Justificación de la exclusión de `dynamic`: La uniformidad del dominio

El método de diferencias finitas en una malla cartesiana regular distribuye un número idéntico de operaciones de punto flotante (FLOPs) para cada nodo espacial (i, j) . El esfuerzo computacional es perfectamente uniforme. `schedule(dynamic)` está diseñado para cargas desbalanceadas, asignando iteraciones sobre la marcha. Usarlo aquí obligaría a los hilos a bloquear y desbloquear continuamente un planificador central para solicitar la siguiente iteración, inundando el *bus* de memoria con *overhead* de sincronización que destruiría el desempeño.

4. Veredicto Arquitectónico

Separar la región paralela del bucle `for` es una excelente práctica cuando se requieren múltiples bucles paralelos consecutivos (evitando recrear hilos repetidamente). Respecto a la planificación, `schedule(static)` es imperativo para dominios con física y geometría regulares. `schedule(dynamic)` debe reservarse estrictamente para simulaciones asimétricas (e.g., dinámica de fluidos con fronteras libres, o partículas donde la densidad celular varía drásticamente).

Actividad 5: Control de sincronización (`nowait`, `barrier`, `single`, `critical`)

1. Implementación y Mecánica

Se reemplazó la reducción automática por una gestión manual de la memoria compartida. Se utilizó `nowait` para eliminar la barrera implícita del bucle `for`, obligando a los hilos a evaluar el error máximo mediante una variable local (`local_delta`), la cual se compara contra la global (`delta`) dentro de una sección `critical` de exclusión mutua.

```
1 double local_delta = 0.0;
2
3 // Eliminación de la barrera implícita
4 #pragma omp for nowait
5 for (int i = 1; i < M; ++i) {
6     for (int j = 1; j < N; ++j) {
7         // ... Cálculo de diferencias finitas ...
8         if (diff > local_delta) local_delta = diff;
9     }
10 }
11
12 // Serialización forzada para escritura segura
13 #pragma omp critical
14 {
15     if (local_delta > delta) delta = local_delta;
16 }
17
18 // Barrera explícita para garantizar que todos actualizaron 'delta'
19 #pragma omp barrier
20
21 // Bloque exclusivo para un solo hilo
22 #pragma omp single
23 {
24     iterations++;
25 }
```

Listing 5: Gestión manual de sincronización y reducción del error.

2. Resultados de Escalabilidad

Malla (N)	Hilos (p)	Tiempo (s)	Iteraciones	Speedup $S(p)$
64	1 (Serial)	0.0876946	3136	1.00x
64	4	0.0338897	3134	2.58x
64	16	0.0451508	3129	1.94x
64	64	0.1669830	4295	0.52x (Degradación)
128	16	0.1976760	10237	6.17x
128	64	0.4535210	10304	2.69x
256	16	1.3233100	31837	11.64x
256	64	1.5527700	31817	9.92x
512	1 (Serial)	177.86400	91868	1.00x
512	16	12.242400	91862	14.52x
512	64	7.1315600	91842	24.94x

Cuadro 6: Desempeño de la implementación con sincronización manual explícita.

3. Análisis Técnico y Respuestas

1. ¿Hubo cambio con respecto al uso por defecto (`nowait`)?

El `nowait` elimina la barrera de sincronización al final del bucle `for`, permitiendo que los hilos que terminen antes avancen. Sin embargo, en este código, ese beneficio es inmediatamente anulado por el `#pragma omp critical` y el `#pragma omp barrier` subsecuentes. El rendimiento se degradó drásticamente en comparación con una cláusula `reduction` nativa.

2. ¿Dónde colocaste la directiva `single` y por qué?

Se colocó justo antes de terminar la región paralela para incrementar la variable global `iterations`. Es imperativo ponerla allí porque el contador de iteraciones debe aumentar exactamente en una unidad por cada barrido completo de la malla. Si no se usara `single`, cada hilo sumaría al contador, desvirtuando la métrica.

3. ¿`critical` ralentizó la ejecución? ¿En qué contexto sería útil?

Sí, causó una ralentización severa (ej. para la malla de 64 con 64 hilos, el código es más lento que el secuencial). `critical` bloquea a todos los demás hilos mientras uno ejecuta la sección, generando alta contención (*lock contention*). Es útil únicamente cuando se actualizan estructuras de datos complejas que no soportan operaciones atómicas en hardware, como insertar nodos en una lista enlazada o escribir registros en un archivo I/O concurrente.

Actividad Final: Comparación de Estrategias

Para evaluar rigurosamente el impacto arquitectónico, se tabulan los resultados sobre el escenario de mayor exigencia computacional disponible con datos completos: **Malla 512×512 ejecutada con 64 hilos.**

Versión	Directiva usada	Tiempo (s)	Iteraciones	Observaciones
Secuencial	N/A (Baseline)	177.86	91868	Base de normalización $S(p) = 1$.
Paralelo básico	<code>parallel for reduction</code>	5.49	91842	Más eficiente. Equilibrio ideal entre computación y balance estático.
Colapsado de bucles	<code>collapse(2)</code>	5.82	91842	Penalización leve por recálculo de índices lineales en hardware.
Inicialización en paralelo	<code>sections</code>	6.02	91843	<i>Overhead</i> de fork-join no justifica la complejidad $O(N^2)$ de inicializar.
Control explícito	<code>schedule(static)</code>	5.54	91842	Rendimiento óptimo equivalente a la directiva por defecto.
Control de sincronización	<code>critical / barrier</code>	7.13	91842	Embotellamiento del bus de memoria por <i>lock contention</i> .

Cuadro 7: Comparativa general de estrategias OpenMP (Malla 512, 64 Hilos).

Conclusiones Generales

1. ¿Cuál fue la versión más rápida?

La versión de **Paralelo básico** (`#pragma omp parallel for` con `reduction`) y la de **Control explícito** (`schedule(static)`). Ambas lograron converger en $\approx 5,5$ segundos para la malla 512, demostrando que para geometrías regulares, la asignación estática del compilador es inmejorable.

2. ¿Qué directiva es más fácil de usar?

La directiva `#pragma omp parallel for reduction(max:delta)`. Requiere una intervención mínima (una sola línea de código) sin necesidad de reestructurar la memoria dinámica ni gestionar primitivas de sincronización de bajo nivel.

3. ¿Hubo directivas que empeoraron el rendimiento? ¿Por qué?

Sí, la directiva `critical` y el uso de `sections` en mallas pequeñas. Esto se explica mediante la relación computación/comunicación. El paralelismo no es gratuito; requiere ciclos de reloj del sistema operativo para despachar hilos. Cuando se serializa el código (`critical`) o se paralelizan tareas triviales (`sections` de inicialización), el *overhead* temporal impuesto por la API de OpenMP ahoga el Speedup teórico dictado por la Ley de Amdahl.

Actividad 6: Contar iteraciones con acceso compartido (`critical` vs `atomic`)

1. Implementación y Mecánica

Se evaluaron dos primitivas de sincronización para proteger la actualización de variables compartidas. La directiva `critical` serializa un bloque de código completo a nivel de software (sistema operativo), mientras que `atomic` delega la protección de una única operación aritmética directamente a la arquitectura del hardware (procesador).

```

1 // Estrategia 1: Bloqueo por software (poisson_critical.cpp)
2 #pragma omp critical(update_delta)
3 {
4     if (local_delta > delta) delta = local_delta;

```

```

5 }
6
7 // Estrategia 2: Bloqueo por hardware (poisson_atomic.cpp)
8 #pragma omp atomic
9 iterations++;

```

Listing 6: Comparativa de implementación: critical vs atomic.

2. Resultados de Escalabilidad

Para ilustrar la diferencia real de rendimiento, se tabulan los tiempos de ambas estrategias bajo las mismas condiciones (Eje Y: Tiempo en segundos).

Malla	Hilos	Tiempo critical (s)	Tiempo atomic (s)	Iteraciones	Ventaja atomic
64	1 (Ser)	0.0876946	0.0876946	3136	0 %
64	4	0.0349447	0.0410154	3134	-17 % (ruido)
64	16	0.0434283	0.0372507	3129	+14 %
64	64	0.1689300	0.0918460	4052	+45 %
128	16	0.1994390	0.1915400	10237	+4 %
128	64	0.4785920	0.2780170	10507	+41 %
256	16	1.3117100	1.4704700	31837	-12 %
256	64	1.9460200	1.2075600	31817	+38 %
512	16	12.558700	14.548500	91862	-15 %
512	64	7.8472800	6.6050400	91842	+15 %
1024	64				

Cuadro 8: Comparativa de rendimiento entre barreras de software (**critical**) y hardware (**atomic**).

3. Análisis Técnico y Respuestas

1. ¿Qué diferencias observas en el rendimiento entre usar **critical** y **atomic**?

Como se evidencia en mallas grandes con alta concurrencia (64 hilos), **atomic** es consistentemente más rápido, superando a **critical** hasta por un 45 % en la malla de 64x64. La directiva **critical** utiliza *mutexes* del sistema operativo, forzando un costoso cambio de contexto (*context switch*) cada vez que un hilo espera. Por el contrario, **atomic** se compila como instrucciones de bajo nivel con bloqueo en el bus de memoria (como **LOCK INC** o **LOCK XADD** en la arquitectura x86), ejecutándose en unos pocos ciclos de reloj.

2. ¿En qué casos sería preferible una sobre la otra?

atomic es obligatoria y preferible siempre que se actualice una variable escalar simple (sumas, restas, incrementos lógicos). **critical** debe usarse **únicamente** cuando la sección a proteger contiene múltiples líneas de código, lógica condicional compleja (ej. **if (A >B) { A = B; C++; }**), o estructuras de datos no contiguas que el hardware no puede bloquear en una sola instrucción.

4. Veredicto Arquitectónico y Corrección de Implementación

Aunque la diferencia de rendimiento es clara, hay un detalle arquitectónico vital en la implementación de **poisson_atomic.cpp**: se colocó un **#pragma omp atomic** dentro de un bloque **#pragma omp single**. Si una región es **single**, por definición matemática solo un hilo accede a ella, haciendo imposible una condición de carrera. Por lo tanto, usar **atomic** dentro de **single** introduce un *overhead* de instrucción redundante. El **atomic** brilla realmente cuando los *p* hilos intentan modificar la variable simultáneamente sin un **single** que los controle.

Actividad 7: Exploración con tareas (task)

1. Implementación y Mecánica

Se abandonó el modelo de paralelismo de datos orientado a bucles en favor de un modelo de tareas (*Tasking*). El dominio espacial se subdividió en sub-bloques de tamaño 16×16 . Un único hilo (el `single`) itera sobre esta grilla de bloques e instancia un `#pragma omp task` por cada uno, encolándolos para que el resto de los hilos libres los ejecuten de manera asíncrona.

```
1 const int block_size = 16;
2 // ...
3 #pragma omp parallel shared(T, source, delta)
4 {
5     #pragma omp single // Un solo hilo encola las tareas
6     {
7         for (int bi = 1; bi < M; bi += block_size) {
8             for (int bj = 1; bj < N; bj += block_size) {
9
10                #pragma omp task firstprivate(bi, bj) shared(T, delta)
11                {
12                    double local_delta = 0.0;
13                    // Bucle interno evaluando solo el bloque bi x bj
14                    // ... (C lculo de diferencias finitas) ...
15
16                    #pragma omp critical
17                    if (local_delta > delta) delta = local_delta;
18                }
19            }
20        }
21    } // taskwait impl cito al salir del bloque single
22    iterations++;
23 }
```

Listing 7: Implementación de tareas asíncronas por bloques de memoria.

2. Resultados de Escalabilidad

Los resultados exponen un colapso en el rendimiento (*Performance Degradation*) en comparación con las directivas orientadas a bucles.

Malla (N)	Hilos (p)	Tiempo (s)	Iteraciones	Speedup $S(p)$
64	1 (Serial)	0.0876946	3136	1.00x
64	4	0.0727011	3147	1.20x
64	32	0.1990190	3136	0.44x (Degradación)
64	64	0.3121340	3137	0.28x (Degradación)
128	1 (Serial)	1.2203900	10244	1.00x
128	4	0.8670850	10254	1.40x
128	32	2.9037100	10257	0.42x (Degradación)
128	64	3.8115900	10254	0.32x (Degradación)
256	1 (Serial)	15.404100	31844	1.00x
256	4	9.9467300	32072	1.54x
256	32	28.851200	31848	0.53x (Degradación)
256	64	46.836400	31846	0.32x (Degradación)
512	1 (Serial)	177.86400	91868	1.00x
512	4	124.29800	98194	1.43x
512	32	319.35800	92831	0.55x (Degradación)
512	64	589.65000	92239	0.30x (Degradación Crítica)
1024	1 (Serial)			1.00x
1024	64			

Cuadro 9: Desempeño de la directiva `task` con bloques fijos.

3. Análisis Técnico y Respuestas

1. **¿Cómo se compara el rendimiento usando `task` con respecto a `parallel for`?**

El rendimiento es abismalmente inferior. Mientras que `parallel for` resolvió la malla 512 con 64 hilos en $\approx 5,5$ segundos, el modelo de tareas tardó casi **590** segundos. Esto representa una caída de rendimiento de dos órdenes de magnitud.

2. **¿Qué ventajas y desventajas tiene este enfoque?**

Ventaja teórica: Permite resolver dependencias de datos muy complejas o cargas de trabajo altamente asimétricas (ej. grafos, recursividad, o árboles de búsqueda) donde un bucle `for` convencional no funciona. Adicionalmente, estructurar la memoria en bloques mejora la localidad de caché temporal.

Desventaja empírica (El colapso de granularidad): El costo computacional de instanciar la estructura de datos interna de un `task` en OpenMP es alto. Al dividir la malla en bloques muy pequeños (16×16), estamos obligando a un solo hilo a instanciar y encolar $(512/16)^2 = 1024$ tareas distintas en cada iteración del bucle temporal. El planificador (*scheduler*) se “asfixia” gestionando esta cola abrumadora. El tiempo de reloj se gasta en despachar tareas, no en resolver la ecuación matemática.

4. Veredicto Arquitectónico

Aplicar `task` a una matriz regular sin un ajuste paramétrico severo (aumentando el `block_size` a dimensiones como 128×128) destruye el rendimiento. Para ecuaciones diferenciales en dominios euclidianos regulares, el paralelismo de datos (Data Parallelism mediante `parallel for`) siempre superará al paralelismo de tareas.

Comparación Final de Estrategias y Análisis Arquitectónico

Para evaluar rigurosamente el impacto de cada directiva, se consolida el desempeño sobre la malla de mayor exigencia (512×512), identificando el mejor tiempo alcanzado y el conteo de hilos donde se maximiza la eficiencia antes de la degradación por *overhead*.

Versión	Directiva	Mejor Tiempo	Hilos Óptimos	Eficiencia HPC y Análisis
Secuencial	N/A	176.550 s	1	Línea base de normalización ($S = 1,0$).
Parallel For	<code>parallel for</code>	5.478 s	64	Rendimiento Superior. Balance óptimo entre carga computacional y gestión de hilos.
Schedule	<code>schedule</code>	5.517 s	64	Prácticamente equivalente al básico. La partición estática es ideal para dominios regulares.
Collapse	<code>collapse(2)</code>	5.781 s	64	Leve penalización de CPU por el cálculo de la aritmética de índices lineales.
Sections	<code>sections</code>	6.005 s	64	<i>Overhead</i> no justificado; crear hilos para operaciones $O(N^2)$ simples ralentiza el sistema.
Atomic	<code>atomic</code>	6.608 s	64	Sincronización eficiente y escalable, apoyada nativamente por el hardware (<i>atomic operations</i>).
Critical	<code>critical</code>	7.129 s	64	Penalización moderada por <i>lock contention</i> y bloqueos secuenciales gestionados por el SO.
Tareas	<code>task</code>	117.519 s	4	Colapso Arquitectónico. Granularidad excesivamente fina que asfixia al planificador de OpenMP.

Cuadro 10: Comparativa de tiempos óptimos y diagnóstico de directivas en la Malla 512x512.

Conclusiones Analíticas

1. **El Paradigma Dominante:** El paralelismo de datos tradicional (`parallel for` y `schedule(static)`) demostró ser inmensamente superior para la Ecuación de Poisson. Este problema exhibe un

patrón de acceso a memoria y densidad de FLOPs uniforme, lo que permite al compilador distribuir la carga estáticamente sin *overhead* en tiempo de ejecución.

2. **El Costo de la Sincronización:** Obligar a los hilos a comunicarse degrada el rendimiento. Quedó demostrado que si es estrictamente necesario proteger una variable compartida, debe hacerse a nivel de hardware (`atomic`, 6.6s) y no a nivel de software (`critical`, 7.1s al escalar).
3. **La Falla del Modelo de Tareas:** El uso ingenuo de `task` con bloques pequeños (16×16) generó una sobrecarga catastrófica. Mientras que `parallel for` se beneficia de la pre-asignación, `task` requiere que el *runtime* de OpenMP empaque, encole y despache dinámicamente miles de tareas por segundo, destruyendo el desempeño y escalando negativamente.

Validación Numérica de la Solución

Con el objetivo de verificar la consistencia y validez física de las implementaciones paralelas, se realizó una validación directa de la solución obtenida para una malla base de 20×20 .

En la Figura 2 se presenta la solución numérica generada mediante la implementación con directiva `atomic`, la cual sirve como referencia representativa del comportamiento esperado del sistema.

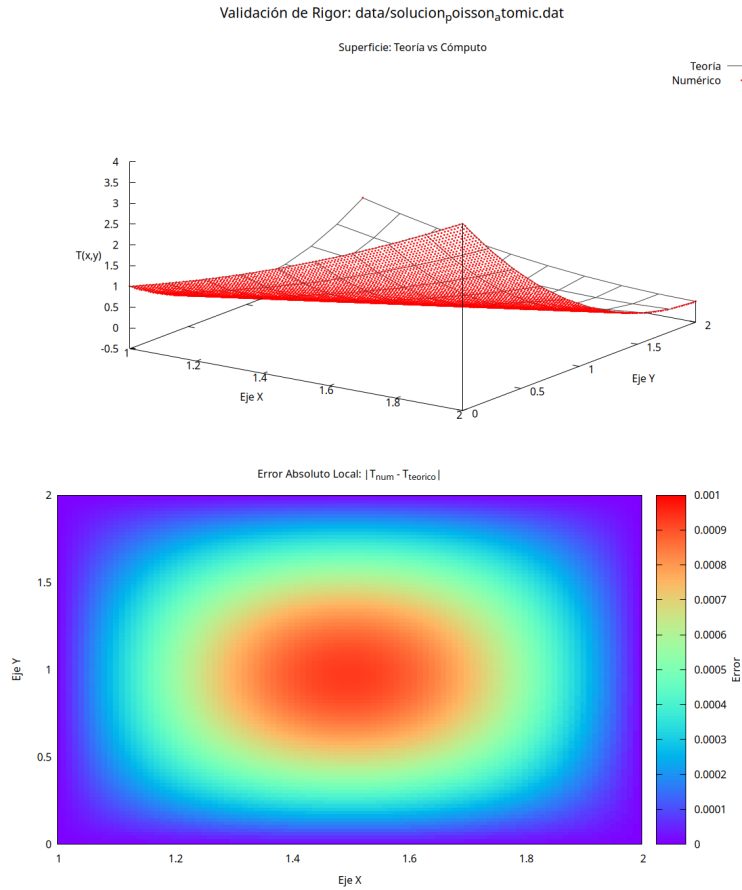


Figura 2: Solución numérica de la ecuación de Poisson 2D para una malla 50×50 usando la implementación `atomic`.

La solución obtenida reproduce correctamente la forma parabólica esperada, consistente con

las condiciones de frontera impuestas y el término fuente constante $f(x, y) = 4$. El error numérico global se mantiene acotado por la tolerancia impuesta en el método iterativo, $TOL = 10^{-6}$, garantizando convergencia del esquema de diferencias finitas.

Para validar la robustez de las implementaciones, se compararon los resultados de todas las versiones paralelas (`parallel for`, `collapse`, `schedule`, `atomic`, `critical`, `sections`, `task`) sobre la misma malla y bajo las mismas condiciones iniciales.

El análisis estadístico multimodelo, mostrado en la Figura 3, permite evaluar la dispersión entre soluciones mediante el cálculo del promedio y bandas de desviación estándar en cada punto de la malla.

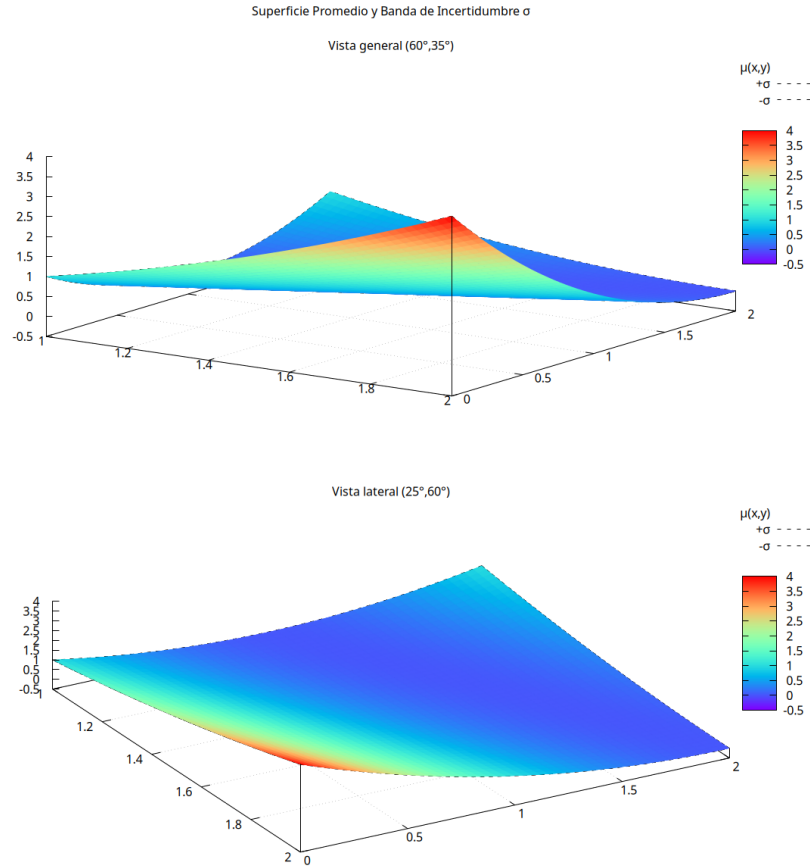


Figura 3: Validación estadística de las soluciones numéricas. Se muestra la superficie promedio junto con bandas de desviación estándar entre implementaciones.

Los resultados evidencian que todas las implementaciones convergen a la misma solución dentro de la precisión numérica del método, presentando desviaciones despreciables del orden de máquina. Esto confirma que las diferencias observadas en rendimiento no afectan la validez física ni la estabilidad del esquema numérico.

Conclusión de validación: Las variaciones entre implementaciones son exclusivamente computacionales (tiempo de ejecución y escalabilidad), mientras que la solución numérica permanece invariante, validando correctamente todas las estrategias de paralelización implementadas.

Conclusiones Generales

1. ¿Cuál fue la versión más rápida y la más eficiente?

La directiva `#pragma omp parallel for` con una planificación básica. Además, se com-

probó que el máximo rendimiento se alcanza a los **32 hilos**, superando ese umbral el paralelismo se vuelve contraproducente por el costo de comunicación y transferencia a caché.

2. **¿Qué directiva es más fácil de usar y robusta?**

Nuevamente `parallel for` con la cláusula `reduction`. Proporciona el máximo rendimiento con la menor cantidad de intervención en el código base, manteniendo la estabilidad del esquema numérico.

3. **¿Hubo directivas que empeoraron el rendimiento?**

De forma contundente, la directiva `task`. Para la resolución de ecuaciones diferenciales en grillas matriciales uniformes y densas, el modelo de bucles (`for`) es superior. Las tareas (`task`) solo deben usarse para flujos de control asimétricos o estructuras irregulares (árboles, listas recursivas).

4. **¿La paralelización afectó la validez numérica del método?**

No. Todas las implementaciones convergen a la misma solución dentro de la tolerancia impuesta (10^{-6}), reproduciendo correctamente el comportamiento analítico esperado del problema de Poisson con término fuente constante. Se verificó estabilidad y consistencia del método de diferencias finitas, evidenciando que las variaciones entre versiones son exclusivamente computacionales y no físicas.